Safety-Preserving Compression Between Generator and Safety Critic: Measuring and Minimizing Safety Loss in Prompt-based PRM

Aidan Pinero
UC Santa Cruz
apinero@ucsc.edu

Glenn Grant-Richards
UC Santa Cruz
ggrantri@ucsc.edu

Sophia Yang
UC Santa Cruz
syang255@ucsc.edu

Abstract

Large language model agents generate verbose reasoning traces that strain computational resources and exceed context window limits of downstream safety critics. We investigate whether compressing these traces before safety evaluation introduces "safety loss"—the degradation in a critic's ability to detect unsafe behaviors. Our pipeline uses entropy-based token pruning (GPT-2) to compress agent outputs before evaluation by Llama Guard 3. Preliminary experiments on AgentDojo (N=10) reveal a surprising finding: compression at ratios $\geq$50% does not cause the critic to miss attacks (0% attack success rate), but instead triggers severe over-refusal, with 90% of content incorrectly flagged as unsafe. This suggests fragmented text from aggressive token pruning produces false safety alarms rather than enabling attacks to slip through undetected, highlighting a critical usability-safety tradeoff for compressed agent systems.

1 Approach

1.1 Generator-Compressor-Critic Pipeline

We implement a three-stage pipeline to measure safety loss under compression. The **Generator** produces structured agent traces containing system prompts, user queries, and conversation history from the AgentDojo dataset (**?**). These traces are fed to an **Entropy-Based Compressor**, which calculates per-token perplexity using GPT-2 (**?**) and selectively removes low-information tokens. Finally, a **Safety Critic** (Llama Guard 3 (**?**)) evaluates the compressed trace and outputs a binary safe/unsafe decision.

1.2 Entropy-Based Compression

Our compressor uses a pre-trained GPT-2 model to compute token-level loss values via cross-entropy. For each token t_i in the input sequence, we calculate:

$$loss(t_i) = -\log P(t_i|t_1, \ldots, t_{i-1})$$

Tokens with higher loss (greater "surprisal") are deemed more informative and retained. Given a compression ratio r, we keep the top $(1-r) \times |T|$ tokens ranked by loss, where $|T|$ is the original sequence length. This approach is similar to methods in LLMLingua (**?**) but operates purely on perplexity without additional filtering heuristics.

1.3 Evaluation Metrics

We measure two key safety metrics:

- **Attack Success Rate (ASR)**: The false negative rate—proportion of unsafe traces incorrectly classified as safe (missed attacks).
- **Benign Refusal Rate (BRR)**: The false positive rate—proportion of safe traces incorrectly classified as unsafe (over-refusal).

1.4 Baselines and Implementation

Our baseline is a standard Generator-Critic pipeline with no compression (compression ratio = 0%). We implemented the pipeline in Python using the Transformers library (**?**) for model loading and inference. All code is original except for the use of pre-trained models (GPT-2, Llama Guard 3) from Hugging Face.

2 Experiments

2.1 Data

We use the AgentDojo benchmark (**?**), a dataset of adversarial attacks targeting LLM-based agents. AgentDojo contains injection attacks embedded in tool outputs and user instructions designed to manipulate agent behavior (e.g., data exfiltration, unauthorized actions). For this milestone, we evaluate on the "tiny" subset (N=10 samples) to validate our pipeline. Each sample includes a system

prompt, user query, and conversation history that forms the agent trace.

2.2 Experimental Details

Models: We use GPT-2 (117M parameters) as the compressor proxy and Llama Guard 3 (8B parameters) as the safety critic. Llama Guard 3 is loaded with `bfloat16` precision and automatic device mapping to fit within GPU memory.

Hardware: All experiments run on a single NVIDIA A100 (80GB) GPU.

Compression Ratios: We test four settings: 0% (baseline), 30%, 50%, and 70% compression.

Procedure: For each sample, we (1) format the agent trace, (2) apply compression at the specified ratio, (3) pass the compressed text to Llama Guard 3, and (4) compare the critic's decision (safe/unsafe) against ground truth labels.

2.3 Results

Table **??** presents our preliminary findings. The baseline (0% compression) achieves perfect classification: no attacks missed (ASR = 0%) and no false alarms (BRR = 0%). Light compression (30%) maintains this performance. However, at 50% and 70% compression, we observe a dramatic spike in over-refusal—BRR jumps to 90%, meaning the critic incorrectly flags 9 out of 10 traces as unsafe, despite zero actual attacks being missed.

Compression	ASR (%)	BRR (%)	Tokens Kept
None (0%)	0.00	0.00	100%
Light (30%)	0.00	0.00	70%
Medium (50%)	0.00	90.00	50%
High (70%)	0.00	90.00	30%

Table 1: Safety metrics across compression ratios on AgentDojo tiny dataset (N=10). ASR = Attack Success Rate (false negatives), BRR = Benign Refusal Rate (false positives).

2.4 Discussion

Contrary to our initial hypothesis, compression did *not* cause the safety critic to miss attacks. Instead, moderate-to-high compression produces fragmented, ungrammatical text that triggers conservative safety judgments. The critic appears to adopt a "refuse when uncertain" policy: when faced with incomprehensible input, it defaults to blocking the content.

Implications: While zero safety loss (ASR = 0%) is positive from a security perspective, a 90% benign refusal rate renders the system unusable in practice. This reveals a critical tradeoff: aggressive compression saves computational resources but degrades user experience through excessive false positives.

Limitations: Our dataset (N=10) is too small for statistically robust conclusions. Additionally, the AgentDojo "tiny" subset appears to contain predominantly benign samples (all traces classified as True Negatives in the baseline), limiting our ability to measure false negatives. We also do not yet have qualitative analysis of *which* linguistic features trigger over-refusal.

3 Future Work

Based on these preliminary findings, we plan to:

(1) Expand to larger datasets: Test on the full AgentDojo benchmark, LLMail-Inject (**?**), and Lakera PINT (**?**) to obtain statistically significant results across diverse attack types and ensure our dataset contains sufficient unsafe examples to properly measure ASR.

(2) Improve compression methods: Implement sentence-level and semantic compression techniques that preserve grammaticality. Token-level pruning creates fragmented text; we hypothesize that maintaining syntactic coherence will reduce false positives while still achieving compression gains.

(3) Safety-aware compression: Train a compressor explicitly optimized to retain safety-critical tokens (e.g., "password", "delete", "transfer") while removing redundant filler words. This could be achieved via supervised learning on labeled safety-relevant spans or reinforcement learning with ASR/BRR as rewards.

(4) Qualitative error analysis: Manually inspect compressed traces that trigger over-refusal to identify linguistic patterns (e.g., missing verbs, broken noun phrases) that cause false alarms. This will inform compression strategies that balance efficiency and interpretability.

(5) Alternative safety critics: Test with other guard models (e.g., Llama Guard 2, Azure AI Content Safety) to determine whether over-refusal under compression is specific to Llama Guard 3's architecture or a general phenomenon across safety classifiers.

References

Marah Abdin, Jyoti Aneja, Hany Awadalla, Ahmed Awadallah, Ammar Ahmad Awan, Nguyen Bach, Amit Bahree, Arash Bakhtiari, Jianmin Bao, Harkirat Behl, Alon Benhaim, Misha Bilenko, Johan Bjorck, Sébastien Bubeck, Martin Cai, Qin Cai, Vishrav Chaudhary, Dong Chen, Dongdong Chen, and 110 others. 2024. Phi-3 technical report: A highly capable language model locally on your phone. *Preprint*, arXiv:2404.14219.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, and 32 others. 2022. Constitutional ai: Harmlessness from ai feedback. *Preprint*, arXiv:2212.08073.

Collin Burns, Pavel Izmailov, Jan Hendrik Kirchner, Bowen Baker, Leo Gao, Leopold Aschenbrenner, Yining Chen, Adrien Ecoffet, Manas Joglekar, Jan Leike, Ilya Sutskever, and Jeff Wu. 2023. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. *Preprint*, arXiv:2312.09390.

Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, and Xia Hu. 2024. Learning to compress prompt in natural language formats. *Preprint*, arXiv:2402.18700.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. *Preprint*, arXiv:2406.13352.

Weizhi Fei, Xueyan Niu, Guoqing Xie, Yingqing Liu, Bo Bai, and Wei Han. 2025. Efficient prompt compression with evaluator heads for long-context transformer inference. *Preprint*, arXiv:2501.12959.

Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context autoencoder for context compression in a large language model. *Preprint*, arXiv:2307.06945.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: Llm-based input-output safeguard for human-ai conversations. *Preprint*, arXiv:2312.06674.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmlingua: Compressing prompts for accelerated inference of large language models. *Preprint*, arXiv:2310.05736.

Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane Luke. 2024. Prompt compression with context-aware sentence encoding for fast and improved llm inference. *Preprint*, arXiv:2409.01227.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *Preprint*, arXiv:2307.03172.

Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. Harmbench: A standardized evaluation framework for automated red teaming and robust refusal. *Preprint*, arXiv:2402.04249.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. 2024. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *Preprint*, arXiv:2403.12968.

Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, Pouya Tafti, Léonard Hussenot, Pier Giuseppe Sessa, Aakanksha Chowdhery, Adam Roberts, Aditya Barua, Alex Botev, Alex Castro-Ros, Ambrose Slone, and 89 others. 2024. Gemma: Open models based on gemini research and technology. *Preprint*, arXiv:2403.08295.

Yunlong Zhao, Haoran Wu, and Bo Xu. 2025. Leveraging attention to effectively compress prompts for long-context llms. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(24):26048–26056.